2026 EDITION

GitLab CI/CD

Complete Guide

From Zero to Production — Pipelines · Security · Kubernetes · AI

50+	16	4	FREE
Pages	Chapters	Sections	Resource

WHAT'S INSIDE

01 Introduction to GitLab CI/CD
02 Architecture & Components
03 Setting Up GitLab
04 .gitlab-ci.yml Deep Dive
05 Jobs, Stages & DAG
06 Variables & Secrets
07 Runners & Autoscaling
08 Artifacts & Caching
09 Dynamic Pipelines
10 Security — SAST, DAST, SCA
11 Testing Strategies
12 Container Registry
13 Kubernetes Deployments
14 GitOps with ArgoCD
15 Monitoring & Rollbacks
16 AI-Powered CI/CD (Duo)

CI/CD Pipelines · Runners · Security · Kubernetes · GitOps · GitLab Duo AI

+ Appendix: CI/CD Variable Reference · Troubleshooting Guide · Node.js, Python & Java Templates

CHAPTER 01

Introduction to GitLab CI/CD

What You'll Learn

In this chapter you'll understand what CI/CD is, why GitLab became the industry's go-to DevOps platform, how it compares to alternatives, and the core philosophy that guides every pipeline you'll ever build.

1.1 What Is CI/CD?

CI/CD stands for Continuous Integration and Continuous Delivery (or Deployment). It is a software engineering practice that automates the process of building, testing, and deploying applications every time a developer pushes code changes. Instead of manually running tests or deploying applications, CI/CD pipelines handle everything automatically, reducing human error and accelerating release cycles dramatically.

Continuous Integration (CI)

Continuous Integration is the practice of automatically integrating code changes from multiple contributors into a shared repository several times a day. Each integration triggers an automated build and test sequence to detect integration errors as quickly as possible.

- Developers push code frequently — multiple times per day
- Every push triggers automated builds and tests
- Broken builds are detected instantly — not days later
- Teams get fast feedback loops on code quality
- Merge conflicts are discovered and resolved early

Continuous Delivery (CD)

Continuous Delivery extends CI by automatically preparing every code change for release to production. The software can be deployed at any time with a single click — the deployment itself may still require human approval but the preparation is fully automated.

- Every passing build produces a deployable artifact
- Staging environments always mirror production
- Release decisions are business decisions, not technical bottlenecks
- Rollbacks are just as easy as deployments

Continuous Deployment

Continuous Deployment goes one step further — every passing build is automatically deployed to production without human intervention. This requires an extremely high degree of test coverage and confidence in your pipeline.

Term	Definition
Continuous Integration	Merge code + build + test automatically
Continuous Delivery	Automated release preparation; manual deploy trigger
Continuous Deployment	Fully automated — deploy to production on every passing build
GitLab Pipeline	Orchestrates all three stages in a single .gitlab-ci.yml file

1.2 Why GitLab?

GitLab started as a Git repository manager but has evolved into a complete DevOps platform — the only single application that covers the entire software development lifecycle. In 2026, GitLab is used by more than 30 million registered users and thousands of enterprises worldwide.

GitLab Advantage

GitLab is the world's most comprehensive DevOps platform. It combines source code management, CI/CD, security scanning, container registry, package registry, monitoring, and AI assistance in ONE tool — eliminating the need for a complex toolchain.

Key Advantages of GitLab

- Single Application — No tool sprawl; one login, one UI, one API
- Built-in CI/CD — No Jenkins, CircleCI, or external orchestrator needed
- Built-in Security — SAST, DAST, dependency scanning, secret detection
- Container & Package Registry — Store Docker images and packages natively
- GitLab Duo (AI) — AI pair programming and pipeline suggestions
- Self-hosted or SaaS — Deploy on your own servers or use GitLab.com
- Open Core Model — Strong open-source community + enterprise features

1.3 GitLab vs Competitors (2026)

Platform	Model	CI/CD Rating
GitLab CI/CD	Built-in, fully integrated	★★★★
GitHub Actions	Built-in, workflow-based	★★★★
Jenkins	External, plugin-heavy	★★★

CircleCI	SaaS only, limited self-host	★★★
Azure DevOps	Microsoft ecosystem bias	★★★★
ArgoCD	GitOps-only (deployment)	★★★

1.4 The DevOps Shack Way

Throughout this guide, we will build real-world pipelines from scratch. Every concept is explained with analogies, diagrams-in-words, and hands-on YAML examples. By the end you will confidently architect, secure, and optimize production-grade GitLab CI/CD pipelines.

CHAPTER 02

GitLab Architecture & Components

Chapter Goal

Understand how GitLab is structured internally — from the web application to the database, runners, and the Gitaly storage layer. This knowledge helps you debug, optimize, and scale your GitLab installation effectively.

2.1 High-Level Architecture

GitLab is composed of several distinct services that work together. Whether you use GitLab.com or self-host, these same components are at work under the hood. Understanding them is essential for troubleshooting pipeline failures and optimizing performance.

Component	Role	Type
GitLab Web (Rails)	The main Ruby on Rails application. Handles all HTTP requests, UI, and API.	Core Service
Gitaly	A gRPC service for Git operations. Replaces direct Git calls.	Storage Layer
PostgreSQL	Primary relational database for all metadata, users, projects.	Database
Redis	Caching, session storage, Sidekiq job queues.	Cache / Queue
Sidekiq	Background job processor for pipelines, emails, webhooks.	Worker
GitLab Runner	Executes CI/CD jobs in isolated environments.	CI Engine
MinIO / Object Store	Stores artifacts, backups, LFS objects.	Blob Storage
NGINX / Workhorse	Reverse proxy and Git smart HTTP handler.	Proxy
GitLab Shell	Handles SSH Git operations.	SSH Gateway
Elasticsearch	Powers advanced global search (Premium).	Search

2.2 GitLab Runner Architecture

The GitLab Runner is a separate open-source project that runs CI/CD jobs. Runners communicate with the GitLab instance via a polling API over HTTPS. They can run on any machine — VMs, bare metal, Kubernetes, or Docker.

How Runners Work

1. Runner polls the GitLab coordinator API every few seconds
2. GitLab coordinator assigns a pending job to the runner
3. Runner downloads the job definition and repository
4. Runner executes each step in the configured executor
5. Runner uploads artifacts and reports status back to GitLab

Runner Types

- Shared Runners — Available to all projects on the instance
- Group Runners — Available to all projects in a group
- Project Runners — Available to a single project only
- Instance Runners (GitLab 16+) — Instance-wide configuration

⚠ Security Note

Shared runners are convenient but may process jobs from untrusted sources. For sensitive workloads (production deployments, secret access), always use dedicated project or group runners that you control.

2.3 GitLab Tiers in 2026

Tier	Key Features	Pricing
Free	Unlimited public/private repos, 400 CI minutes/month	GitLab.com
Premium	Advanced CI features, 10,000 min/month, security policies	$29/user/mo
Ultimate	Full security suite, compliance, AI Duo, Agile planning	$99/user/mo
Self-Managed CE	Community Edition — free, self-hosted	Open Source
Self-Managed EE	Enterprise features on your own infrastructure	License-based

2.4 GitLab Namespaces & Projects

GitLab organizes resources into a hierarchical namespace system. Understanding this hierarchy is critical for configuring CI/CD variables, runners, and access controls.

- Instance → The top level (gitlab.com or your domain)
- Group → Organizational unit (can nest into subgroups 20 levels deep)
- Project → A single repository with its own issues, CI/CD, and registry
- Subgroup → A group within a group (great for microservices monorepos)

Best Practice

Use subgroups to mirror your organizational structure: company → department → team → service. CI/CD variables and runners set at the group level are inherited by all child projects, reducing duplication.

CHAPTER 03

Setting Up GitLab

Chapter Goal

Get GitLab running — whether you choose GitLab.com SaaS for instant start, or self-host on a VM, EC2, or Kubernetes using Omnibus or the Helm chart.

3.1 GitLab.com — Quickest Start

GitLab.com is the fastest way to get started. No infrastructure to manage, automatic upgrades, and instant access to shared runners. Perfect for individuals, startups, and teams that don't need on-premises hosting.

6. Visit https://gitlab.com and click 'Sign up for free'
7. Verify your email address
8. Create your first project or import from GitHub/Bitbucket
9. Pipelines are ready immediately — shared runners are pre-configured

3.2 Self-Hosted with Omnibus (Recommended)

GitLab Omnibus is an all-in-one package that bundles all GitLab components — PostgreSQL, Redis, NGINX, Gitaly, Sidekiq — into a single installation. It is the recommended way to self-host GitLab.

Minimum Requirements (2026)

Resource	Specification
CPU	4 cores (8+ recommended for production)
RAM	8 GB minimum (16 GB recommended)
Storage	50 GB minimum (SSD strongly recommended)
OS	Ubuntu 22.04 LTS / RHEL 9 / Debian 12
Network	Open ports: 80 (HTTP), 443 (HTTPS), 22 (SSH)

Installation on Ubuntu 22.04

```
# Step 1: Install dependencies
sudo apt-get update && sudo apt-get install -y curl openssh-server ca-certificates
```

```
# Step 2: Add GitLab repository
```

```
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh |
sudo bash
```

```
# Step 3: Install GitLab EE (or gitlab-ce for Community Edition)
sudo EXTERNAL_URL='https://gitlab.example.com' apt-get install -y gitlab-ee
```

```
# Step 4: Reconfigure GitLab
sudo gitlab-ctl reconfigure
```

```
# Step 5: Get initial root password
sudo cat /etc/gitlab/initial_root_password
```

Post-Installation

After logging in as root, immediately: (1) Change the root password, (2) Disable new user registration if not needed, (3) Configure SMTP for email notifications, (4) Set up HTTPS with Let's Encrypt, (5) Configure backup schedule.

3.3 Configuring gitlab.rb

The main GitLab configuration file is /etc/gitlab/gitlab.rb. Every setting is managed here. After any change, run gitlab-ctl reconfigure to apply.

```
# /etc/gitlab/gitlab.rb — key settings
external_url 'https://gitlab.example.com'
gitlab_rails['smtp_enable'] = true
gitlab_rails['smtp_address'] = 'smtp.gmail.com'
gitlab_rails['smtp_port'] = 587
gitlab_rails['backup_keep_time'] = 604800  # 7 days
gitlab_rails['lfs_enabled'] = true
puma['worker_processes'] = 4
```

3.4 Kubernetes Install via Helm

For cloud-native self-hosted deployments, GitLab provides an official Helm chart that deploys all components into a Kubernetes cluster with horizontal scaling.

```
# Add GitLab Helm repository
helm repo add gitlab https://charts.gitlab.io/
helm repo update
```

```
# Install GitLab with minimum values
helm upgrade --install gitlab gitlab/gitlab \
  --set global.hosts.domain=example.com \
  --set global.hosts.externalIP=<LB_IP> \
  --set certmanager-issuer.email=admin@example.com \
  --namespace gitlab --create-namespace
```

3.5 Installing GitLab Runner

```
# Install runner on Ubuntu
curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh | sudo bash
sudo apt-get install gitlab-runner
```

```
# Register runner
sudo gitlab-runner register \
  --url https://gitlab.example.com \
  --token <YOUR_RUNNER_TOKEN> \
  --executor docker \
  --docker-image alpine:latest \
  --description 'My Docker Runner'
```

Runner Token (GitLab 16+)

GitLab 16+ replaced the shared registration token with short-lived runner authentication tokens. Generate tokens from Settings → CI/CD → Runners → New runner. This improves security significantly.

CHAPTER 04

Your First Pipeline — .gitlab-ci.yml Deep Dive

Chapter Goal

Master the .gitlab-ci.yml file — the heart of every GitLab CI/CD pipeline. You'll understand every key, write your first real pipeline, and avoid the 10 most common YAML mistakes.

4.1 The .gitlab-ci.yml File

The .gitlab-ci.yml file lives at the root of your repository and defines your entire CI/CD pipeline. GitLab reads this file every time a pipeline is triggered and uses it to determine what jobs to run, in what order, and under what conditions.

Minimal Valid Pipeline

```
# .gitlab-ci.yml — the simplest possible pipeline
stages:
  - build
  - test
  - deploy
```

```
build-job:
  stage: build
  script:
    - echo 'Building the application...'
    - mvn clean package -DskipTests
```

```
test-job:
  stage: test
  script:
    - echo 'Running tests...'
    - mvn test
```

```
deploy-job:
  stage: deploy
  script:
    - echo 'Deploying to production...'
  only:
    - main
```

4.2 Top-Level Keywords

Keyword	Purpose	Required?
stages	Defines the ordered list of stages	Required (or default used)
default	Sets defaults for all jobs	Optional
variables	Defines pipeline-wide variables	Optional
include	Imports other CI/CD files	Optional
workflow	Controls when the pipeline runs	Optional (recommended)
image	Default Docker image for all jobs	Optional
before_script	Commands run before every job	Optional
after_script	Commands run after every job	Optional
services	Additional Docker containers for jobs	Optional
cache	Global cache configuration	Optional

4.3 Job Keywords

Keyword	Purpose	Usage
stage	Assigns job to a stage	Most jobs
script	Shell commands to execute	Required
image	Docker image for this job	Optional
before_script	Commands before script	Optional
after_script	Commands after script	Optional
variables	Job-level variables	Optional
rules	Conditional job execution	Recommended
only/except	Older branch/tag filtering	Legacy
artifacts	Files to preserve after job	Optional
cache	Files to cache between jobs	Optional
dependencies	Artifacts to download	Optional
needs	DAG — run without waiting for stage	Optional
tags	Select specific runners	Optional
when	on_success / on_failure / always	Optional
timeout	Job time limit	Optional
retry	Auto-retry on failure	Optional
parallel	Run N copies in parallel	Optional
environment	Deployment target	Optional

allow_failure	Non-blocking failures	Optional
extends	Inherit from job template	Optional

4.4 The `default` Block

The default block lets you set configuration that applies to all jobs unless overridden. This reduces repetition significantly in large pipelines.

```
# .gitlab-ci.yml — using default block
default:
  image: node:20-alpine
  before_script:
    - npm ci --cache .npm --prefer-offline
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths:
      - .npm/
  retry:
    max: 2
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
```

4.5 Workflow Rules — Control Pipeline Triggers

The workflow keyword determines when an entire pipeline runs (vs individual job rules which control individual jobs). This is critical for avoiding duplicate pipelines.

```
# Avoid duplicate pipelines — best practice 2026
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
    - if: '$CI_COMMIT_TAG'
    - when: never    # Block all other triggers
```

⚠ Common Mistake

Without workflow rules, GitLab creates two pipelines for a merge request: one for the MR event and one for the branch push. The workflow block above prevents this duplication, saving CI minutes and reducing confusion.

4.6 Pipeline Linting

Always validate your .gitlab-ci.yml before pushing. GitLab provides a built-in linter at CI/CD → Pipelines → CI Lint. You can also use the API or CLI tools.

```
# Validate via GitLab API
curl --header 'PRIVATE-TOKEN: <token>' \
  --form 'content=@.gitlab-ci.yml' \
  'https://gitlab.com/api/v4/ci/lint'
```

```
# Validate with gitlab-ci-local (popular open-source tool)
npm install -g @firecow/gitlab-ci-local
gitlab-ci-local --list
gitlab-ci-local build-job
```

CHAPTER 05

Jobs, Stages & Dependencies

Chapter Goal

Master job ordering, stage logic, and GitLab's DAG (Directed Acyclic Graph) feature with `needs`. Learn how to dramatically speed up pipelines by eliminating unnecessary sequential waits.

5.1 Understanding Stages

Stages define the order in which job groups execute. All jobs within the same stage run in parallel by default. A stage does not begin until all jobs in the previous stage complete successfully.

```
# Standard DevOps pipeline stages
stages:
  - validate    # Linting, YAML validation
  - build       # Compile, package
  - test        # Unit + integration tests
  - scan        # Security and quality scans
  - package     # Docker image build
  - deploy-staging
  - test-staging
  - deploy-production
```

5.2 Parallel Jobs Within a Stage

All jobs in the same stage run simultaneously. This is the primary mechanism for parallelism in classic GitLab pipelines. Use it to run independent test suites, linters, and scans concurrently.

```
unit-tests:
  stage: test
  script: npm run test:unit
```

```
integration-tests:
  stage: test
  script: npm run test:integration
```

```
e2e-tests:
  stage: test
  script: npm run test:e2e
```

The three test jobs above all run simultaneously. Total test time = max(unit, integration, e2e) — not the sum.

5.3 DAG Pipelines with `needs`

The needs keyword enables Directed Acyclic Graph (DAG) pipelines. A job with needs does not wait for its stage to complete — it runs as soon as all its needed jobs finish, regardless of stage. This can reduce total pipeline time by 40-60%.

Performance Win

Without DAG: Pipeline time = sum of slowest job in each stage. With DAG: Pipeline time = longest path through the dependency graph. For a 10-stage pipeline, DAG can cut total time from 45 minutes to 18 minutes.

```
# DAG example — deploy-staging starts as soon as build AND unit-test finish
# It does NOT wait for integration-test or scan to complete
build:
  stage: build
  script: make build
  artifacts:
    paths: ['dist/']
```

```
unit-test:
  stage: test
  needs: ['build']
  script: make test-unit
```

```
integration-test:
  stage: test
  needs: ['build']
  script: make test-integration
```

```
deploy-staging:
  stage: deploy-staging
  needs: ['build', 'unit-test']   # starts immediately after these two!
  script: make deploy-staging
```

5.4 Job Dependencies vs Needs

Keyword	Behavior
needs	Controls execution order (DAG) + artifact download
dependencies	Controls artifact download only (no order change)
No keyword	Job downloads artifacts from all previous stage jobs
dependencies: []	Job downloads NO artifacts (explicit empty)

5.5 Parallel Matrix Jobs

The parallel:matrix keyword lets you run the same job with different configurations simultaneously. Perfect for testing across multiple Node.js versions, database backends, or browsers.

```
test-across-versions:
  stage: test
  parallel:
    matrix:
      - NODE_VERSION: ['18', '20', '22']
        BROWSER: ['chrome', 'firefox']
  image: node:${NODE_VERSION}-alpine
  script:
    - npm test -- --browser=${BROWSER}
```

The above creates 6 parallel jobs: node18+chrome, node18+firefox, node20+chrome, node20+firefox, node22+chrome, node22+firefox — all running simultaneously.

5.6 Job Control with `when`

Value	Behavior	Use Case
on_success	Run only if all previous jobs succeeded	Default
on_failure	Run only if at least one previous job failed	Cleanup jobs
always	Run regardless of previous job status	Notifications
manual	Run only when manually triggered in UI	Production deploy
delayed	Run after a delay (e.g., start_in: 5 minutes)	Canary deploys
never	Never run (effectively disables job)	Override rules

CHAPTER 06

Variables, Environments & Secrets

Chapter Goal

Master GitLab CI/CD variables — predefined, custom, masked, and protected. Learn proper secrets management, environment-specific configuration, and how to safely handle credentials in 2026.

6.1 Predefined CI/CD Variables

GitLab automatically injects dozens of variables into every pipeline. These provide context about the current commit, branch, user, pipeline, and project — use them to make your pipeline logic dynamic.

Variable	Description	Example
CI_COMMIT_SHA	Full 40-character commit hash	abc123def456...
CI_COMMIT_SHORT_SHA	First 8 chars of commit SHA	abc123de
CI_COMMIT_BRANCH	Branch name (not in MR pipelines)	main
CI_COMMIT_TAG	Git tag name (tag pipelines only)	v1.2.3
CI_COMMIT_MESSAGE	Full commit message	Fix login bug
CI_PIPELINE_ID	Unique pipeline ID	12345
CI_JOB_ID	Unique job ID	67890
CI_JOB_NAME	Current job name	build-job
CI_PROJECT_NAME	Project name	my-app
CI_PROJECT_PATH	Full project path	group/my-app
CI_REGISTRY	Container registry hostname	registry.gitlab.com
CI_REGISTRY_IMAGE	Full image path for this project	registry.gitlab.com/group/my-app
CI_DEFAULT_BRANCH	Default branch name	main
GITLAB_USER_EMAIL	Email of user who triggered pipeline	dev@company.com
CI_ENVIRONMENT_NAME	Current environment name	production

6.2 Custom Variables

You can define custom variables at three levels: in the .gitlab-ci.yml file, in the GitLab UI (Settings → CI/CD → Variables), or via the API. Variables defined in the UI override those in the file.

Variables in YAML

```yaml
variables:
  APP_VERSION: '1.0.0'
  DOCKER_DRIVER: overlay2
  MAVEN_OPTS: '-Xmx1g -XX:MaxPermSize=256m'
```

```yaml
build-job:
  stage: build
  variables:
    BUILD_ENV: production   # Job-level overrides global
  script: echo $BUILD_ENV
```

Variable Precedence (Highest to Lowest)

10. Manual pipeline variables (triggered via API or UI)
11. Scheduled pipeline variables
12. Project CI/CD variables (set in UI)
13. Group CI/CD variables (inherited from parent group)
14. Instance CI/CD variables (set by GitLab admin)
15. Job-level variables in .gitlab-ci.yml
16. Global variables in .gitlab-ci.yml

6.3 Masked & Protected Variables

Flag	Effect
Masked	Value is hidden in job logs — appears as [MASKED]
Protected	Only available in protected branches/tags
Masked + Protected	Maximum security — production secrets
File type	Value is written to a temp file; variable contains the path

Secret Best Practices 2026

1. NEVER put secrets in .gitlab-ci.yml. 2. Always mask production credentials. 3. Mark production variables as Protected + Masked. 4. Rotate secrets quarterly. 5. Use HashiCorp Vault or AWS Secrets Manager for enterprise-grade secrets management.

6.4 Environments

Environments let you track deployments to different targets (staging, production, QA). GitLab maintains a deployment history per environment and shows which version is currently live.

```yaml
deploy-production:
```

```
  stage: deploy-production
  script:
    - helm upgrade --install my-app ./chart
  environment:
    name: production
    url: https://myapp.com
    on_stop: stop-production
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+/'
```

6.5 Vault Integration (2026 Best Practice)

GitLab 15+ supports native HashiCorp Vault integration via JWT authentication. Jobs can fetch secrets from Vault without storing them in GitLab at all.

```
fetch-db-password:
  stage: build
  id_tokens:
    VAULT_ID_TOKEN:
      aud: https://vault.example.com
  secrets:
    DB_PASSWORD:
      vault: production/data/database#password
  script:
    - echo 'DB password retrieved from Vault'
```

CHAPTER 07

GitLab Runners — Executors & Auto-scaling

Chapter Goal

Deep dive into GitLab Runner executors, when to use each one, how to configure them, and how to auto-scale runners on AWS, GCP, and Kubernetes for cost-efficient, high-throughput CI/CD.

7.1 Runner Executors

An executor is the mechanism a runner uses to run jobs. Choosing the right executor is one of the most important architecture decisions for your CI/CD infrastructure.

Executor	How It Works	Best For
Docker	Runs each job in a fresh Docker container	Most common; good isolation
Kubernetes	Runs each job in a Kubernetes pod	Cloud-native; auto-scales
Shell	Runs jobs directly on the runner host	Simple; low isolation
Docker Machine	Spins up cloud VMs on-demand	AWS/GCP auto-scaling
VirtualBox	Runs jobs in VirtualBox VMs	Desktop testing
Parallels	Runs jobs in Parallels VMs (macOS)	macOS CI
SSH	Connects to remote host via SSH	Legacy systems
Custom	Define your own executor via scripts	Specialized needs

7.2 Docker Executor Configuration

```
# /etc/gitlab-runner/config.toml — Docker executor
[[runners]]
  name = 'docker-runner'
  url = 'https://gitlab.com'
  token = 'YOUR_TOKEN'
  executor = 'docker'
  [runners.docker]
    image = 'alpine:latest'
    privileged = false
    volumes = ['/cache']
    shm_size = 0
```

```
    pull_policy = ['if-not-present', 'always']
    disable_cache = false
```

⚠ Privileged Mode Warning

Setting privileged = true allows Docker-in-Docker (DinD) but removes container isolation completely. In 2026, prefer Kaniko or Buildah for container builds instead of DinD to avoid privileged mode entirely.

7.3 Kubernetes Executor

The Kubernetes executor is the preferred choice for cloud-native CI/CD. Each job runs in its own Kubernetes pod, providing complete isolation and enabling massive horizontal scaling.

```
# Kubernetes executor config
[[runners]]
  name = 'k8s-runner'
  executor = 'kubernetes'
  [runners.kubernetes]
    host = ''  # Uses in-cluster config
    namespace = 'gitlab-runners'
    service_account = 'gitlab-runner-sa'
    cpu_request = '100m'
    memory_request = '128Mi'
    cpu_limit = '1'
    memory_limit = '1Gi'
    poll_interval = 3
    poll_timeout = 180
```

7.4 Auto-scaling with Fleeting (2026)

GitLab Fleeting is the next-generation instance auto-scaling technology replacing the deprecated Docker Machine. It provisions cloud instances (AWS EC2, GCP Compute) on demand and terminates them after job completion.

```
# Fleeting plugin configuration for AWS EC2
[[runners]]
  executor = 'docker-autoscaler'
  [runners.autoscaler]
    plugin = 'fleeting-plugin-aws'
    capacity_per_instance = 4
    max_use_count = 1
    max_instances = 20
    [runners.autoscaler.plugin_config]
      name = 'gitlab-fleeting-group'
      region = 'us-east-1'
    [runners.autoscaler.policy]
      idle_count = 2
      idle_time = '30m0s'
```

7.5 Runner Tags & Routing

Use runner tags to route specific jobs to specific runners. This is essential for routing deployment jobs to runners with production credentials, or build jobs to runners with high CPU.

```
build-heavy:
  stage: build
  tags:
    - high-cpu
    - linux
  script: make build
```

```
deploy-prod:
  stage: deploy-production
  tags:
    - production-runner
    - aws-us-east-1
  script: helm upgrade --install my-app ./chart
```

CHAPTER 08

Artifacts, Caching & Optimization

Chapter Goal

Understand the difference between artifacts and cache, configure each correctly, and apply proven optimization strategies to cut your pipeline times by 50% or more.

8.1 Artifacts vs Cache

Feature	Artifacts vs Cache
Artifacts	Passed between jobs/stages in the same pipeline
Cache	Persisted across pipeline runs (e.g., node_modules)
Artifacts	Uploaded to GitLab after job completes
Cache	Stored on runner or distributed cache backend
Artifacts	Example: compiled binaries, test reports, Docker images
Cache	Example: npm packages, Maven repository, pip packages
Artifacts	Downloaded by default by all downstream jobs
Cache	Not automatically shared — must be explicitly configured

8.2 Configuring Artifacts

```
build-app:
  stage: build
  script: npm run build
  artifacts:
    name: '${CI_JOB_NAME}-${CI_COMMIT_SHORT_SHA}'
    paths:
      - dist/
      - build/
    exclude:
      - dist/**/*.map    # Exclude source maps
    expire_in: 1 week
    when: always    # Keep artifacts even on failure
```

Test Reports as Artifacts

GitLab can parse test reports (JUnit XML, coverage reports) and display results directly in merge requests — no external tools needed.

```
test:
  stage: test
  script: pytest --junitxml=report.xml --cov=app --cov-report xml:coverage.xml
  coverage: '/TOTAL.*\s+(\d+%)$/'   # Regex to extract coverage %
  artifacts:
    reports:
      junit: report.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    expire_in: 30 days
```

8.3 Caching Strategies

```
# Cache node_modules — key based on package-lock.json hash
cache:
  key:
    files:
      - package-lock.json
  paths:
    - node_modules/
  policy: pull-push   # pull on start, push on completion
```

```
# Separate read and write cache jobs for better performance
install-deps:
  stage: build
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths: ['node_modules/']
    policy: push   # Only write cache
  script: npm ci
```

```
run-tests:
  stage: test
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths: ['node_modules/']
    policy: pull   # Only read cache (faster)
  script: npm test
```

8.4 Pipeline Optimization Checklist

1. Use needs: for DAG parallelism — biggest single win
2. Set cache with content-hash keys (files: [package-lock.json])
3. Use pull-only cache policy in read-heavy jobs
4. Exclude unnecessary files from artifacts
5. Set expire_in on all artifacts — don't store forever
6. Use interruptible: true on MR jobs — cancel outdated pipelines

7. Use rules: instead of only/except for precision
8. Break monolithic jobs into smaller parallel jobs
9. Use shallow clones: GIT_DEPTH: 10 for faster checkout
10. Pin Docker image tags — avoid latest to prevent cache misses

Quick Win

Add `GIT_DEPTH: 10` as a global variable to enable shallow git clones. For large repositories, this alone can reduce git checkout time from 45 seconds to 3 seconds — with zero pipeline logic changes.

CHAPTER 09

Dynamic & Conditional Pipelines

Chapter Goal

Build intelligent pipelines that adapt to their context — run different jobs for MRs vs. main branch, skip unnecessary stages, dynamically generate pipeline stages at runtime, and use include for modular CI/CD.

9.1 Rules — The Modern Conditional

Rules replaced the older only/except syntax. Rules evaluate conditions in order and apply the first match. They give you fine-grained control over when a job runs, and whether it runs automatically, manually, or not at all.

```
deploy-production:
  stage: deploy-production
  script: helm upgrade --install my-app ./chart
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
      when: manual
      allow_failure: false
    - if: '$CI_COMMIT_BRANCH == "main" && $FORCE_DEPLOY == "true"'
      when: on_success
    - when: never
```

Rules with Changes

The changes: keyword makes a job run only when specified files are modified. Perfect for monorepos where you only want to rebuild the service that actually changed.

```
build-frontend:
  stage: build
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      changes:
        - frontend/**/*
        - package.json
  script: npm run build:frontend
```

9.2 Include — Modular Pipelines

The include keyword allows you to split your pipeline into multiple files. This is essential for large projects, monorepos, and organizations that want to standardize CI/CD across multiple projects.

```
# Main .gitlab-ci.yml — orchestrator
include:
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/test.yml'
```

```
  - local: '.gitlab/ci/security.yml'
  - project: 'devops-shack/ci-templates'
    ref: main
    file: '/templates/docker-build.yml'
  - template: Security/SAST.gitlab-ci.yml
  - remote: 'https://example.com/ci/shared.yml'
```

Include Types

Type	Source	Example
local	File in the same repository	'.gitlab/ci/test.yml'
project	File from another GitLab project	'group/project' + file
template	GitLab built-in template	Security/SAST.gitlab-ci.yml
remote	File from public URL	https://...yaml
component	CI/CD Component (new in 2024)	gitlab.com/catalog/...

9.3 CI/CD Components (2025/2026)

CI/CD Components are the new, standardized way to share reusable pipeline configurations. They are versioned, documented, and discoverable via the GitLab CI/CD Catalog — similar to npm packages for pipelines.

```
# Use a component from the GitLab Catalog
include:
  - component: gitlab.com/gitlab-org/components/sast@1.0.0
  - component: gitlab.com/devops-shack/components/docker-build@2.1.0
    inputs:
      image_name: $CI_REGISTRY_IMAGE
      push_to_registry: 'true'
```

9.4 Dynamic Child Pipelines

Dynamic child pipelines allow you to generate pipeline YAML at runtime and trigger it as a child pipeline. This is the ultimate tool for monorepos — generate a different pipeline per changed service.

```
generate-pipeline:
  stage: build
  script:
    - python scripts/generate_pipeline.py > generated-pipeline.yml
  artifacts:
    paths: [generated-pipeline.yml]
```

```
trigger-child:
  stage: deploy-staging
  needs: [generate-pipeline]
  trigger:
    include:
      - artifact: generated-pipeline.yml
        job: generate-pipeline
    strategy: depend
```

CHAPTER 10

Security Scanning — SAST, DAST, SCA & IaC

Chapter Goal

Implement a complete DevSecOps pipeline with GitLab's built-in security scanners: SAST (code), DAST (runtime), Dependency Scanning (SCA), Secret Detection, Container Scanning, and IaC scanning.

10.1 DevSecOps Philosophy

DevSecOps integrates security into every stage of the CI/CD pipeline — shift security left so vulnerabilities are caught at code commit time, not in production. GitLab is unique in providing ALL of these scanners built into the platform — no third-party integrations needed.

Security Scanning Stack 2026

Scanner	What It Does	Catches
SAST	Static Application Security Testing	Finds code vulnerabilities
DAST	Dynamic Application Security Testing	Tests running application
Dependency Scan	Software Composition Analysis (SCA)	Finds vulnerable dependencies
Secret Detection	Finds leaked API keys, tokens	Prevents credential exposure
Container Scan	Scans Docker images for CVEs	Secures container supply chain
IaC Scan	Scans Terraform, CloudFormation, K8s	Finds infra misconfigurations
License Scan	Identifies open source licenses	Compliance management
API Security	Tests REST/GraphQL APIs (Ultimate)	API vulnerability detection
Coverage-guided Fuzzing	Fuzzes application inputs (Ultimate)	Finds edge case bugs

10.2 Enabling Security Scanners

GitLab provides official templates for each scanner. Simply include them in your pipeline.

```
# Complete DevSecOps pipeline — include all scanners
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/DAST.gitlab-ci.yml
```

```
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml
  - template: Security/SAST-IaC.gitlab-ci.yml
  - template: Security/License-Scanning.gitlab-ci.yml
```

```
variables:
  SAST_EXCLUDED_PATHS: 'spec,test,tests,tmp'
  DS_EXCLUDED_PATHS: 'spec,test'
  CONTAINER_SCANNING_IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  DAST_WEBSITE: https://staging.myapp.com
  DAST_FULL_SCAN_ENABLED: 'false'
```

10.3 Security Policies

GitLab Security Policies (available in Ultimate) enforce security scans at the group level — no individual project can opt out. This is the governance layer for enterprise DevSecOps.

Policy Type	Enforcement
Scan Execution Policy	Forces specific security scans to run on all pipelines
Scan Result Policy	Blocks MR merging when critical vulnerabilities are found
Merge Request Approval	Requires security team approval for high-severity findings
Software License Policy	Blocks use of specific open-source licenses

10.4 Blocking Pipelines on Vulnerabilities

```
# Custom job to fail pipeline if critical CVEs found
check-vulnerabilities:
  stage: scan
  image: python:3.12-slim
  needs: [container_scanning, dependency_scanning]
  script:
    - pip install gitlab-security-report-parser
    - parse-report gl-dependency-scanning-report.json --max-severity high
  allow_failure: false
```

Zero Tolerance Policy

In production pipelines for 2026, set allow_failure: false on your security jobs. Any critical or high vulnerability should block the deployment. Use exceptions judiciously via GitLab vulnerability dismissal with comments.

CHAPTER 11

Testing Strategies

Chapter Goal

Design a comprehensive testing strategy in GitLab CI/CD: unit tests, integration tests, end-to-end tests, performance tests, and chaos engineering — with results visible directly in merge requests.

11.1 Testing Pyramid in CI/CD

The testing pyramid guides how to allocate testing effort. Unit tests are cheap and fast — run thousands of them. E2E tests are slow and expensive — run fewer but on critical paths only.

Test Type	What It Tests	Frequency
Unit Tests	Test individual functions/classes in isolation	Fastest — run on every commit
Integration Tests	Test multiple components together	Medium — run on every commit
Contract Tests	Test API interfaces between services	Medium — run on every MR
E2E Tests	Test complete user flows in a browser	Slow — run on main/release
Performance Tests	Load and stress testing	Slow — run on release branches
Chaos Tests	Inject failures to test resilience	Weekly or on-demand

11.2 Unit Tests with Coverage

```
unit-test-node:
  stage: test
  image: node:20-alpine
  script:
    - npm ci
    - npm run test:unit -- --coverage
  coverage: '/Statements.*?:\s+(\d+\.?\d*)%/'
  artifacts:
    reports:
      junit: junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```

11.3 Integration Tests with Services

GitLab services spin up additional Docker containers alongside your job container. Perfect for running tests against a real database or message queue.

```
integration-test:
  stage: test
  image: python:3.12
  services:
    - name: postgres:16
      alias: db
    - name: redis:7-alpine
      alias: cache
  variables:
    POSTGRES_DB: testdb
    POSTGRES_USER: testuser
    POSTGRES_PASSWORD: testpass
    DATABASE_URL: postgresql://testuser:testpass@db:5432/testdb
  script:
    - pip install -r requirements.txt
    - python -m pytest tests/integration/
```

11.4 E2E Testing with Playwright

```
e2e-tests:
  stage: test-staging
  image: mcr.microsoft.com/playwright:v1.41.0-focal
  variables:
    BASE_URL: https://staging.myapp.com
  script:
    - npm ci
    - npx playwright test --reporter=junit
  artifacts:
    reports:
      junit: playwright-report/results.xml
    paths:
      - playwright-report/
    expire_in: 7 days
```

11.5 Review Apps

Review Apps automatically deploy your MR branch to a unique URL so reviewers can test changes in a real environment before merging. GitLab manages the lifecycle — environment is created on MR open, destroyed on MR close.

```
deploy-review:
  stage: deploy-staging
  script:
    - helm upgrade --install review-$CI_MERGE_REQUEST_IID ./chart
    - echo 'https://review-$CI_MERGE_REQUEST_IID.example.com'
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://review-$CI_MERGE_REQUEST_IID.example.com
    on_stop: stop-review
  rules:
    - if: '$CI_MERGE_REQUEST_IID'
```

CHAPTER 12

Container Registry & Docker Workflows

Chapter Goal

Master building, tagging, scanning, and pushing Docker images through GitLab CI/CD using the built-in GitLab Container Registry. Learn Kaniko for rootless builds and multi-architecture image builds.

12.1 GitLab Container Registry

GitLab includes a built-in Docker Container Registry at no extra cost. Images are stored per-project and access is controlled by GitLab permissions. No external registry like Docker Hub or ECR needed.

Concept	Details
Registry URL	registry.gitlab.com/<group>/<project>
Auth with CI	Use $CI_REGISTRY_USER and $CI_REGISTRY_PASSWORD
Image naming	$CI_REGISTRY_IMAGE:<tag>
Cleanup policy	Configure in Settings → Packages → Cleanup policies
Storage	Backed by object storage (MinIO/S3/GCS)

12.2 Docker Build Pipeline (2026 Best Practice)

The recommended approach in 2026 is to use Kaniko for Docker builds. Kaniko builds images without requiring Docker daemon access, eliminating the privileged container security risk entirely.

```
# Secure Docker build with Kaniko — no privileged mode needed
build-image:
  stage: package
  image:
    name: gcr.io/kaniko-project/executor:v1.23.0-debug
    entrypoint: ['']
  script:
    - mkdir -p /kaniko/.docker
    - echo
'{"auths":{"$CI_REGISTRY":{"username":"$CI_REGISTRY_USER","password":"$CI_REGISTRY_PAS
SWORD"}}}' > /kaniko/.docker/config.json
    - /kaniko/executor
      --context $CI_PROJECT_DIR
      --dockerfile $CI_PROJECT_DIR/Dockerfile
```

```
      --destination $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
      --destination $CI_REGISTRY_IMAGE:latest
      --cache=true
      --cache-repo=$CI_REGISTRY_IMAGE/cache
```

12.3 Image Tagging Strategy

Tag Pattern	Meaning	When to Use
$CI_COMMIT_SHORT_SHA	Immutable — every commit gets unique tag	Always use
latest	Mutable — points to most recent build	Development only
$CI_COMMIT_TAG	Semantic version tag (v1.2.3)	Production releases
$CI_COMMIT_BRANCH	Branch name tag	Branch testing
$CI_ENVIRONMENT_SLUG	Environment-based tag	Environment deploys

12.4 Multi-Architecture Builds

In 2026, supporting ARM64 alongside AMD64 is critical — AWS Graviton, Apple Silicon, and ARM-based cloud instances are mainstream. GitLab CI can build multi-arch images using BuildKit.

```
build-multi-arch:
  stage: package
  image: docker:26
  services:
    - docker:26-dind
  variables:
    DOCKER_BUILDKIT: '1'
  script:
    - docker buildx create --use --name multiarch
    - docker buildx build --platform linux/amd64,linux/arm64 \
        --push -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
```

12.5 Registry Cleanup Policy

Unmanaged registries fill up fast. Configure automatic cleanup in GitLab UI or via API to remove old image tags and reclaim storage.

11. Navigate to Settings → Packages & Registries → Container Registry
12. Enable cleanup policy
13. Set name patterns (e.g., keep tags matching /^v\d+.\d+.\d+$/)
14. Set retention period (e.g., keep last 10 tags)
15. Run policy weekly or daily

CHAPTER 13

Kubernetes Deployments with GitLab

Chapter Goal

Deploy applications to Kubernetes clusters through GitLab CI/CD using Helm, kubectl, and the GitLab Agent for Kubernetes (GitLab KAS). Master deployment strategies: rolling, blue/green, and canary.

13.1 GitLab Agent for Kubernetes (KAS)

The GitLab Agent for Kubernetes (KAS) is the modern, secure way to connect a Kubernetes cluster to GitLab. It replaces the older certificate-based integration and works without any network exposure — the agent connects outbound to GitLab.

Installing the GitLab Agent

```
# Install GitLab Agent in your cluster via Helm
helm repo add gitlab https://charts.gitlab.io
helm upgrade --install gitlab-agent gitlab/gitlab-agent \
  --namespace gitlab-agent \
  --create-namespace \
  --set config.token=<AGENT_TOKEN> \
  --set config.kasAddress='wss://kas.gitlab.com'
```

```
# Agent configuration — .gitlab/agents/my-agent/config.yaml
ci_access:
  projects:
    - id: group/my-project
  groups:
    - id: my-group
```

13.2 Deploying with Helm

```
deploy-staging:
  stage: deploy-staging
  image: dtzar/helm-kubectl:3.14
  variables:
    KUBECONFIG: $KUBE_CONFIG_STAGING
  script:
    - kubectl config use-context $KUBE_CONTEXT
    - helm upgrade --install my-app ./helm/my-app \
        --namespace staging \
        --set image.repository=$CI_REGISTRY_IMAGE \
        --set image.tag=$CI_COMMIT_SHORT_SHA \
        --set replicaCount=2 \
```

```
        --wait --timeout 5m0s
  environment:
    name: staging
    url: https://staging.myapp.com
```

13.3 Deployment Strategies

Rolling Deployment (Default)

Kubernetes replaces pods gradually — a percentage of old pods are replaced with new ones at a time. Zero downtime, simple to configure.

```
# values.yaml — rolling update strategy
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

Blue/Green Deployment

Two identical environments (blue and green) exist simultaneously. Traffic is switched from blue to green instantly. If issues arise, switch back to blue immediately.

16. Deploy new version (green) alongside current version (blue)
17. Run smoke tests against green environment
18. Switch load balancer to point to green (instant cutover)
19. Monitor for 30 minutes — roll back by switching back to blue
20. Decommission blue environment after confirmation

Canary Deployment

Send a small percentage of traffic (5-10%) to the new version while most users stay on the old version. Gradually increase the canary percentage as confidence grows.

```
deploy-canary:
  stage: deploy-staging
  script:
    - helm upgrade my-app-canary ./helm/my-app \
        --set image.tag=$CI_COMMIT_SHORT_SHA \
        --set replicaCount=1 \          # 1 of 10 pods = 10% traffic
        --set service.enabled=false \   # Use main service
        --set ingress.weight=10
```

13.4 Post-Deployment Verification

```
smoke-test:
  stage: test-staging
  needs: [deploy-staging]
  script:
    - sleep 30  # Wait for pods to be ready
    - curl -sf https://staging.myapp.com/health || exit 1
    - curl -sf https://staging.myapp.com/api/version | grep $CI_COMMIT_SHORT_SHA
```

CHAPTER 14

GitOps with GitLab & ArgoCD

Chapter Goal

Implement GitOps — the practice of using Git as the single source of truth for both application code and infrastructure state. Connect GitLab CI/CD with ArgoCD for automated, auditable Kubernetes deployments.

14.1 GitOps Principles

GitOps is a deployment methodology where the desired state of your entire system is stored in Git. Any change to the infrastructure or application configuration is made through a Git commit, which triggers automated reconciliation.

Principle	What It Means	How GitLab Enables It
Declarative	Describe desired state, not imperative steps	Kubernetes manifests
Versioned	Every change is a Git commit with history	git log, git diff
Automated	System automatically reconciles drift	ArgoCD sync loop
Approved	All changes go through MR + code review	GitLab MR workflow

14.2 GitOps Repository Structure

Separate your application code repo from your GitOps (infrastructure) repo. The CI/CD pipeline in the app repo builds and pushes the image, then creates an MR in the GitOps repo to update the image tag.

```
# GitOps repository structure
gitops-repo/
├── apps/
│   ├── staging/
│   │   ├── my-app/
│   │   │   ├── deployment.yaml
│   │   │   └── kustomization.yaml
│   └── production/
│       └── my-app/
│           ├── deployment.yaml
│           └── kustomization.yaml
├── infrastructure/
│   ├── cert-manager/
│   └── ingress-nginx/
```

```
└── clusters/
    ├── staging.yaml
    └── production.yaml
```

14.3 CI Pipeline Updates GitOps Repo

After building and pushing the Docker image, the CI pipeline automatically creates an MR in the GitOps repo with the new image tag. This gives you a full audit trail and human review before production deployment.

```
update-gitops-repo:
  stage: deploy-staging
  image: alpine/git:latest
  script:
    - git clone https://gitlab-ci-token:$CI_TOKEN@gitlab.com/org/gitops-repo.git
    - cd gitops-repo
    - git config user.email 'ci@devopsshack.com'
    - git config user.name 'GitLab CI'
    - git checkout -b update-$CI_PROJECT_NAME-$CI_COMMIT_SHORT_SHA
    - sed -i 's|image: .*my-app:.*|image: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA|'
apps/staging/my-app/deployment.yaml
    - git add apps/staging/my-app/deployment.yaml
    - git commit -m 'Update my-app to $CI_COMMIT_SHORT_SHA'
    - git push origin update-$CI_PROJECT_NAME-$CI_COMMIT_SHORT_SHA
    - glab mr create --fill --web
```

14.4 ArgoCD Configuration

ArgoCD watches the GitOps repository and automatically syncs the cluster state to match what's declared in Git. Configure ArgoCD Applications to point to your GitOps repo.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-staging
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://gitlab.com/org/gitops-repo.git
    targetRevision: main
    path: apps/staging/my-app
  destination:
    server: https://kubernetes.default.svc
    namespace: staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

CHAPTER 15

Monitoring, Observability & Rollbacks

Chapter Goal

Integrate monitoring and observability into your GitLab CI/CD pipeline — automated rollbacks, deployment metrics, alerting, and linking deployments to performance dashboards.

15.1 GitLab Environments & Deployments

GitLab maintains a complete deployment history per environment. You can view which commit is deployed where, who deployed it, when, and roll back with a single click in the UI.

Feature	What It Shows	Benefit
Deployments page	Shows full history of deploys per environment	Audit trail
Environment dashboard	Current version deployed per environment	Overview
One-click rollback	Re-run previous deployment job	Quick recovery
Stop environment	Spin down review apps or staging	Cost saving
Deployment frequency	DORA metric — how often you deploy	Velocity KPI

15.2 Automated Rollback Strategy

```
deploy-production:
  stage: deploy-production
  script:
    - helm upgrade --install my-app ./chart \
        --set image.tag=$CI_COMMIT_SHORT_SHA \
        --atomic --wait --timeout 5m0s
  environment: {name: production, url: https://myapp.com}
```

```
smoke-test-production:
  stage: verify
  script:
    - sleep 30
    - ./scripts/smoke-test.sh https://myapp.com || ./scripts/rollback.sh
```

Automatic Rollback Script

```
# scripts/rollback.sh
```

```
#!/bin/bash
echo 'Smoke test FAILED — initiating rollback!'
PREV_TAG=$(helm history my-app -n production --max 2 -o json |
  jq -r '.[0].chart' | cut -d'-' -f2)
helm rollback my-app 0 -n production --wait
echo 'Rolled back to previous version'
exit 1  # Fail the pipeline to notify team
```

15.3 DORA Metrics in GitLab

GitLab natively tracks the four DORA (DevOps Research and Assessment) metrics — the industry standard for measuring DevOps performance. Access them at Analytics → CI/CD Analytics.

DORA Metric	Description & Target (Elite)
Deployment Frequency	How often you deploy to production — aim for multiple times/day
Lead Time for Changes	Time from code commit to production — aim for < 1 hour
Change Failure Rate	% of deployments causing incidents — aim for < 5%
Time to Restore Service	How long to recover from failures — aim for < 1 hour

15.4 Integrating Prometheus & Grafana

Connect GitLab environments to Prometheus to see deployment markers on your Grafana dashboards. GitLab sends a deployment event to Prometheus Alertmanager via webhook.

```
notify-grafana:
  stage: verify
  script:
    - curl -X POST $GRAFANA_WEBHOOK_URL \
        -H 'Content-Type: application/json' \
        -d '{"text":"Deployed $CI_PROJECT_NAME:$CI_COMMIT_SHORT_SHA to production by
$GITLAB_USER_EMAIL"}'
```

15.5 Incident Response Pipeline

When an alert fires, GitLab can automatically create an incident, page the on-call engineer via PagerDuty, and trigger a diagnostic pipeline — all from a single webhook.

Closed-Loop Incident Management

GitLab 2026 introduces AI-powered incident triage: when a deployment causes an alert, GitLab Duo automatically analyzes recent commits, identifies the likely cause, and suggests the rollback command — reducing MTTR by up to 70%.

CHAPTER 16

AI-Powered CI/CD — GitLab Duo & 2026 Trends

Final Chapter

Explore how AI is transforming CI/CD in 2026 — GitLab Duo features, AI-generated pipelines, intelligent test selection, predictive failure detection, and where the industry is heading.

16.1 GitLab Duo — AI Across the DevOps Lifecycle

GitLab Duo is GitLab's suite of AI features integrated throughout the platform. Unlike external AI tools, Duo understands your code, your pipeline history, your security vulnerabilities, and your deployment patterns.

Feature	Description	Where Used
Duo Code Suggestions	AI pair programmer in your IDE	VS Code, JetBrains, vim
Duo Chat	Conversational AI assistant in GitLab	Web UI, IDE plugin
Duo Code Review	AI reviews MRs, suggests improvements	Merge Request widget
Duo Root Cause Analysis	AI diagnoses pipeline failures	Failed job view
Duo Vulnerability Explanation	AI explains security findings	Security dashboard
Duo Suggested Remediation	AI fixes security vulnerabilities	One-click apply
Duo Test Generation	AI writes unit tests for your code	IDE and Web
Duo CI Config Suggestions	AI generates .gitlab-ci.yml	Pipeline editor

16.2 AI-Generated Pipelines

In GitLab 17+ (2026), the Pipeline Editor includes AI assistance. Describe your pipeline in plain English and Duo generates the YAML. It understands your project language, framework, and testing tools automatically.

Example Duo Prompt

"I have a Node.js Express API with PostgreSQL. Generate a pipeline with: linting with ESLint, unit tests with Jest and code coverage, Docker build with Kaniko, SAST scanning, and Helm deploy to staging on merge to main, and production on git tag." → Duo generates complete .gitlab-ci.yml in seconds.

16.3 Root Cause Analysis (RCA)

When a pipeline fails, GitLab Duo analyzes the job logs, related test output, recent commits, and similar historical failures to explain why the job failed and suggest a fix — all within the GitLab UI.

21. Job fails — GitLab displays 'Explain with AI' button in the failed job view
22. Click 'Explain with AI' — Duo analyzes logs, recent code changes, and historical patterns
23. Duo explains the root cause in plain English
24. Duo suggests specific code or configuration changes to fix the issue
25. Developer applies the suggestion — pipeline succeeds

16.4 Intelligent Test Selection (ITS)

Running the full test suite on every commit is wasteful. Intelligent Test Selection analyzes code changes and test coverage data to run only the tests most likely to catch regressions — reducing test time by 40-80%.

```
smart-test:
  stage: test
  script:
    - gitlab-test-optimizer analyze --coverage coverage.json
    - pytest $(cat optimized-test-list.txt)
```

16.5 The Future of CI/CD (2026 and Beyond)

Trend	Description	Timeline
AI Pipeline Generation	Natural language → working CI/CD YAML	Available now in GitLab Duo
Predictive Failure Detection	ML models predict job failures before they happen	GitLab 17.x
Self-Healing Pipelines	AI auto-retries with modified config on failure	2025-2026
Autonomous Security Response	AI auto-creates MRs to fix CVEs	GitLab 17.x Ultimate
Green CI/CD	Carbon-aware scheduling of CI workloads	Emerging 2026
AI Deployment Approval	AI validates deployment safety before promotion	Early 2026
Voice-Driven DevOps	Voice commands to trigger pipelines	Experimental

16.6 GitLab 2026 Platform Roadmap Highlights

- GitLab 18.0 — Unified AI platform with Duo Enterprise tier

- Pipeline Efficiency Score — AI scores your pipeline and suggests optimizations
- Cross-project pipeline analytics — Org-wide CI/CD health dashboard
- GitLab Cells — Horizontal scaling for massive GitLab instances
- Compliance frameworks 2.0 — AI-generated compliance evidence
- GitLab Remote Development — Full cloud IDE with integrated CI/CD
- AI code quality gates — AI blocks merges that degrade code quality

Congratulations!

You have completed the DevOps Shack GitLab CI/CD Complete Guide 2026! You now have the knowledge to design, build, secure, optimize, and operate production-grade GitLab CI/CD pipelines. Join the DevOps Shack community @devopsshack for daily tips, new content, and live sessions!

APPENDIX — Quick Reference

Complete CI/CD Variable Reference

Variable	Description	Example Value
CI_API_V4_URL	GitLab API v4 URL	https://gitlab.com/api/v4
CI_BUILDS_DIR	Directory where builds are stored	/builds
CI_COMMIT_AUTHOR	Author of the commit	John <j@c.com>
CI_COMMIT_BEFORE_SHA	Previous commit SHA	abc123...
CI_COMMIT_DESCRIPTION	Description part of commit message	Detailed description
CI_COMMIT_REF_NAME	Branch or tag name	main / v1.2.3
CI_COMMIT_REF_PROTECTED	Is this a protected branch/tag?	true / false
CI_COMMIT_REF_SLUG	URL-safe ref name	my-feature-branch
CI_COMMIT_TIMESTAMP	Commit timestamp (ISO 8601)	2026-01-01T00:00:00+00:00
CI_COMMIT_TITLE	Title of the commit message	Fix login bug
CI_CONCURRENT_ID	Job index in the runner	0
CI_CONFIG_PATH	Path to CI config file	.gitlab-ci.yml
CI_DEPLOY_FREEZE	Is pipeline in a deploy freeze?	true / false
CI_ENVIRONMENT_SLUG	URL-safe environment name	production
CI_ENVIRONMENT_TIER	Tier of environment	production/staging/testing
CI_JOB_STAGE	Stage of current job	build
CI_JOB_STATUS	Current status of the job	running
CI_JOB_TOKEN	Token for authenticating with GitLab API	glpat-...
CI_JOB_URL	URL of the job	https://gitlab.com/.../jobs/123
CI_MERGE_REQUEST_IID	MR IID (not global ID)	42
CI_MERGE_REQUEST_SOURCE_BRANCH	Source branch of the MR	my-feature

CI_MERGE_REQUEST_TARGET_BRANCH	Target branch of the MR	main
CI_PIPELINE_SOURCE	What triggered the pipeline	push/merge_request_event
CI_PIPELINE_URL	URL of the pipeline	https://gitlab.com/.../pipelines/789
CI_PROJECT_DIR	Absolute path of the project	/builds/group/project
CI_PROJECT_ID	Numeric project ID	12345
CI_PROJECT_NAMESPACE	Project namespace	my-group
CI_PROJECT_ROOT_NAMESPACE	Top-level group	my-org
CI_PROJECT_URL	HTTP(S) address of project	https://gitlab.com/group/proj
CI_PROJECT_VISIBILITY	Project visibility	public/internal/private
CI_REGISTRY_USER	Username for GitLab registry	gitlab-ci-token
CI_REGISTRY_PASSWORD	Password for GitLab registry	(CI job token value)
CI_RUNNER_DESCRIPTION	Description of the runner	My Docker Runner
CI_RUNNER_EXECUTABLE_ARCH	Runner architecture	amd64 / arm64
CI_RUNNER_ID	Runner's numeric ID	789
CI_RUNNER_REVISION	Runner version	16.9.0
CI_RUNNER_TAGS	Runner tags	['docker','linux']
CI_SERVER_HOST	Host of GitLab instance	gitlab.com
CI_SERVER_VERSION	GitLab version	17.5.0

Pipeline Troubleshooting Guide

Problem	Likely Cause	Solution
Job stuck in pending	No runner matches tags/availability	Check runner tags, verify runner is online
YAML syntax error	Malformed .gitlab-ci.yml	Use CI Lint tool, validate with gitlab-ci-local
Cache not working	Wrong cache key or paths	Check cache key matches between jobs, verify paths exist
Artifact not found	Job didn't produce artifact	Check script output, ensure path is correct
Docker: permission denied	Privileged mode not enabled	Enable privileged on runner or switch to Kaniko
needs: job not found	Referenced job doesn't exist	Check job name spelling, ensure job is in YAML
Pipeline not triggered on MR	workflow:rules blocking it	Review workflow rules, check pipeline source condition
Environment not updating	Environment name mismatch	Verify environment: name matches exactly across jobs
Secret not available	Variable not set or not protected	Check variable scope, protected/masked settings
Container pull fails	Registry auth issue	Check CI_REGISTRY_USER/PASSWORD, image path
Helm deploy fails	Chart values incorrect or timeout	Add --debug flag, check pod events in cluster
SAST scanner fails	Unsupported language/version	Check GitLab SAST language support matrix
Coverage not showing	Regex doesn't match output	Test regex with actual job log content
MR pipeline skipped	rules: none matched	Add fallback rule or check condition logic

Pipeline Templates Gallery

Node.js Full-Stack Template

```
# Complete Node.js CI/CD — DevOps Shack Template
default:
  image: node:20-alpine
  cache:
    key: {files: [package-lock.json]}
    paths: [node_modules/]
```

```
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
    - if: '$CI_COMMIT_TAG'
    - when: never
```

```
stages: [install, lint, test, build, scan, package, deploy-staging, deploy-prod]
```

```
install:
  stage: install
  script: npm ci
  cache: {policy: push}
```

```
lint:
  stage: lint
  needs: [install]
  script: [npm run lint, npm run type-check]
```

```
unit-test:
  stage: test
  needs: [install]
  script: npm run test:unit -- --coverage
  coverage: '/Statements.*:(\d+\.\d+)%/'
  artifacts: {reports: {junit: junit.xml}}
```

```
build:
  stage: build
  needs: [unit-test, lint]
  script: npm run build
  artifacts: {paths: [dist/], expire_in: '1 week'}
```

Python/Django Template

```
default:
  image: python:3.12-slim
  before_script:
    - pip install -r requirements.txt --quiet
```

```
test:
  stage: test
  services: [{name: postgres:16, alias: db}]
  variables: {DATABASE_URL: postgresql://user:pass@db/testdb}
  script:
    - pytest --cov=app --cov-report=xml --junitxml=report.xml
  artifacts:
    reports:
      junit: report.xml
      coverage_report: {coverage_format: cobertura, path: coverage.xml}
```

Java/Maven Template

```
default:
  image: maven:3.9-eclipse-temurin-21
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths: [.m2/repository/]
```

```
build:
  stage: build
  script: mvn clean package -DskipTests
  artifacts: {paths: [target/*.jar]}
```

```
test:
  stage: test
  script: mvn test
  artifacts:
    reports: {junit: target/surefire-reports/*.xml}
```

Thank You from DevOps Shack!

Stay Connected

Follow @devopsshack on Instagram and LinkedIn for daily DevOps content, new guides, live sessions, and community support. Hit save, share with your team, and let's build better pipelines together!

Instagram	LinkedIn
@devopsshack	@devopsshack

© 2026 DevOps Shack — Aditya. All rights reserved.

Built with for the DevOps community